

独立Linuxサーバー構築・移行手順書

origin = 内部予定表、tamanegi = 外部空き状況。どちらもUbuntu Server 24.04 LTSを対象とします。作業PCにある02_releaseの完成済みファイルを転送する方法が基本です。サーバー上のビルドは不要です。

本書のJSON → SQLiteという操作は「復元」と呼びます。日常バックアップの開始方法と、故障時の復元・確認・切替・切戻しを後半に掲載しました。

章	内容
1～3	準備・完成物転送・必要ソフト
4～8	origin配置、設定、PHP、Nginx、TLS
9～11	tamanegi配置・設定・PHP・Nginx
12～14	社内入口・初期化・移行・連携確認
15～16	再送・監視・毎日22時バックアップ
17～21	JSONからの復元、検証、世代調整、切替、再開・切戻し
22～24	さくらの実設定、保守、受入・障害対応
25	専門用語をやさしく説明

注意：DNS名・証明書・社内ネットワーク範囲・SSH管理者は組織によって異なります。これらは事前に管理担当者から受け取ってください。未確定値を推測して実行せず、エラーが出た段階で止めます。

1 開始前の確認と記入シート

本書は専用の2台を想定します。既存サイトを同居させる場合、NginxやPHPの停止は他サイトに影響します。停止範囲は管理者へ確認してください。originを社外へ公開しません。

本書の表記	事前に決める値
ADMIN@origin / ADMIN@tamanegi	SSH用管理者名と接続先。以下のコマンド中で置き換える。
ORIGIN_FQDN / TAMANEKI_FQDN	正式なWeb用ホスト名。例のまま使わない。
社内の接続元	社内LAN/VPNからoriginだけへ接続を許す範囲。
証明書と秘密鍵	各ホスト用の証明書一式と秘密鍵のパス。組織管理者から受領。
2種類の共有秘密	社内ポータル↔origin用CBC鍵と、origin↔tamanegi用HMAC鍵。別々の値を使う。
停止時間と復旧担当	利用者への周知、設定・元DBの安全な保管場所。

コマンド欄はBash系シェルで実行します。行末のバックスラッシュ (\) は次の行へ続く印です。sudo nanoで編集するときはCtrl+O→Enterで保存、Ctrl+Xで閉じます。秘密値をPDFやGitHubへ記録しないでください。

新規構築は3章から順番に実施。既存予定を移す場合は本番利用開始前に13章も実施。JSONから復元する場合は17章から読み、対象ファイルが本書のバックアップ形式であることを確認します。

2 作業PC：完成済みファイルを転送

GitHubから承認済みmainを入手し、リポジトリの最上位で実行します。初回は空の作業フォルダーを用意してください。以下ではサーバーのホームに一時置場を作り、既存公開先へ直接上書きしません。

```
git clone https://github.com/Apfelengineer/260725_GW_WEB.git
cd 260725_GW_WEB
git log -1 --oneline
cd 02_release
shasum -a 256 -c SHA256SUMS
cd ..
ssh ADMIN@origin 'mkdir -p ~/kptc-upload/origin ~/kptc-upload/deploy'
ssh ADMIN@tamanegi 'mkdir -p ~/kptc-upload/tamanegi'
rsync -av 02_release/origin/ ADMIN@origin:~/kptc-upload/origin/
rsync -av 01_source/deploy/ ADMIN@origin:~/kptc-upload/deploy/
rsync -av 02_release/tamanegi/ ADMIN@tamanegi:~/kptc-upload/tamanegi/
scp 02_release/SHA256SUMS ADMIN@origin:~/kptc-upload/
scp 02_release/SHA256SUMS ADMIN@tamanegi:~/kptc-upload/
```

正常なら照合がすべてOK、転送がエラーなく終了します。Linuxの作業PCではshasum -a 256をsha256sumに読み替えます。WindowsはWSL等のBash環境を使用してください。

01_source/deployはOSの設定例6ファイル等を渡すためのものです。アプリ本体は02_releaseを使用します。renkonは本番配置不要です。再ビルドが必要な開発時だけ、01_sourceでpnpm install --frozen-lockfile、pnpm run check、pnpm test、pnpm run buildを行います。

3 両サーバー：必要ソフトと時刻

originとtamanegiにSSHで入り、各サーバーで実行します。既存Webサーバーがある場合は競合を確認してから導入してください。

```
sudo apt update
sudo apt install -y nginx php8.3-fpm php8.3-cli php8.3-sqlite3 \
  php8.3-curl php8.3-mbstring sqlite3 rsync curl
sudo timedatectl set-timezone Asia/Tokyo
sudo timedatectl set-ntp true
sudo systemctl enable --now nginx php8.3-fpm
systemctl is-active nginx php8.3-fpm
php -v
php -m | grep -Ei 'sqlite3|pdo_sqlite|openssl|curl|mbstring'
timedatectl status
```

正常なら両サービスがactive、PHP 8.3、sqlite3・pdo_sqlite・openssl・curl・mbstringが表示されます。tamanegiはSQLite DBを使用しませんが、手順を共通化するため同じパッケージを入れています。時刻同期が有効であることも確認します。

Nginxはページの受付係、PHP-FPMはPHPの実行係です。ブラウザでindex.htmlを直接開く方式ではPHPは動きません。

参照：Ubuntu公式 PHP導入説明 <https://ubuntu.com/server/docs/programming-php/>
Ubuntu 24.04リリース情報 <https://documentation.ubuntu.com/release-notes/24.04/>

4 origin : 完成物と非公開領域の配置

originで実行。kptc-20260904を新規構築用の版名とします。同じ版のフォルダーが既にある場合は、未使用の版名へ変更してください。

```
cd ~/kptc-upload
grep '^.* origin/' SHA256SUMS | sha256sum -c -
sudo install -d -m 755 /opt/kptc-scheduler/internal/releases
sudo mkdir /opt/kptc-scheduler/internal/releases/kptc-20260904
sudo rsync -a origin/ /opt/kptc-scheduler/internal/releases/kptc-20260904/
sudo chown -R root:www-data /opt/kptc-scheduler/internal/releases/kptc-20260904
sudo find /opt/kptc-scheduler/internal/releases/kptc-20260904 \
  -type d -exec chmod 750 {} \;
sudo find /opt/kptc-scheduler/internal/releases/kptc-20260904 \
  -type f -exec chmod 640 {} \;
sudo ln -s /opt/kptc-scheduler/internal/releases/kptc-20260904 \
  /opt/kptc-scheduler/internal/current
sudo install -d -o www-data -g www-data -m 700 /var/lib/kptc-scheduler
sudo install -d -o www-data -g www-data -m 700 /var/lib/kptc-scheduler/backups
sudo install -d -o root -g www-data -m 750 /etc/kptc-scheduler
ls -l /opt/kptc-scheduler/internal/current
```

正常ならすべての照合がOK、currentが今回の版を指します。lnで「既に存在」が出たら上書きせず、更新作業として旧版の行先を記録してから23章に従います。DB・JSON・秘密設定はcurrentの外へ置きます。

5 origin : 非公開設定ファイル

originで `sudo nano /etc/kptc-scheduler/internal-env.php` を開き、以下を保存します。ホスト名と2種類の鍵は置き換えてください。鍵はそれぞれ `openssl rand -hex 32` で生成し、対応する相手側だけへ同じ値を設定します。

```
<?php
putenv('KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher.sqlite');
putenv('KPTC_SCHEDULER_BACKUP_JSON=/var/lib/kptc-scheduler/backups/scheduler-latest.json');
putenv('KPTC_PORTAL_TOKEN_KEY=REPLACE_PORTAL_SECRET');
putenv('KPTC_PUBLIC_AVAILABILITY_MODE=https');
putenv('KPTC_PUBLIC_AVAILABILITY_ENDPOINT=https://TAMANEGI_FQDN/receive-availability.php');
putenv('KPTC_PUBLIC_AVAILABILITY_PAGE_URL=https://TAMANEGI_FQDN/');
putenv('KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_PUBLISH_SECRET');
putenv('KPTC_PUBLIC_AVAILABILITY_TIMEOUT=10');
putenv('KPTC_SESSION_COOKIE_SECURE=1');
```

```
sudo chown root:www-data /etc/kptc-scheduler/internal-env.php
sudo chmod 640 /etc/kptc-scheduler/internal-env.php
sudo php -l /etc/kptc-scheduler/internal-env.php
```

正常ならNo syntax errors。設定値はブラウザーやログに出さないでください。Web用PHPと定期実行は同じ設定を読む構成にし、DB切替時の変更漏れを防ぎます。

KPTC_PORTAL_TOKEN_KEYは社内サイト側と共有。KPTC_PUBLIC_AVAILABILITY_SECRETはtamanegi側と共有 (32 文字以上)。この2つを混同しないでください。

6 origin : PHP-FPMの専用設定

originで `sudo nano /etc/php/8.3/fpm/pool.d/kptc-origin.conf` を開き、以下を保存します。

```
[kptc-origin]
user = www-data
group = www-data
listen = /run/php/php8.3-fpm-kptc-origin.sock
listen.owner = www-data
listen.group = www-data
pm = ondemand
pm.max_children = 8
pm.process_idle_timeout = 10s
clear_env = yes
env[KPTC_INTERNAL_CONFIG_FILE] = /etc/kptc-scheduler/internal-env.php
php_admin_value[session.save_path] = /var/lib/kptc-scheduler/sessions
```

```
sudo install -d -o www-data -g www-data -m 700 /var/lib/kptc-scheduler/sessions
sudo php-fpm8.3 -t
sudo systemctl restart php8.3-fpm
ls -l /run/php/php8.3-fpm-kptc-origin.sock
```

正常なら設定検査がsuccessfulで、専用socketが表示されます。sessionsはこのアプリ専用に使います。障害復旧時に他サイトのログイン状態を巻き込まず、新しい空の保存先へ切り替えられます。

PHPファイルを静的ファイルとして配信すると秘密や内部処理が漏れます。次章で許可する入口だけをPHPとして実行してください。

7 origin : NginxとHTTPS

正式証明書を管理者に用意してもらい、`/etc/ssl/kptc/origin-fullchain.pem`と`origin-key.pem`へ安全に配置します。秘密鍵はrootのみ読める600にします。以下は専用ホストの例で、URLは `https://ORIGIN_FQDN/` です。

`sudo nano /etc/nginx/sites-available/kptc-origin` で保存：

```
server {
    listen 80;
    server_name ORIGIN_FQDN;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name ORIGIN_FQDN;
    ssl_certificate /etc/ssl/kptc/origin-fullchain.pem;
    ssl_certificate_key /etc/ssl/kptc/origin-key.pem;
    root /opt/kptc-scheduler/internal/current;
    index index.php;
    access_log off;
    location / { try_files $uri $uri/ /index.php?$query_string; }
    location ~ ^/(index|api)\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm-kptc-origin.sock;
    }
    location ~ \.php$ { return 404; }
    location ~ /\. { deny all; }
}
```

入口は`index.html`ではなく`index.php`です。`access_log off`はURLのtokenが通常のアクセスログへ残ることを避ける例です。上流プロキシやブラウザー履歴等に残る可能性は別途対策が必要です。

8 origin : 公開範囲と動作確認

ネットワーク管理者と、originの443を社内LAN/VPNだけから許可する設定を先に行います。SSHは管理端末だけに制限します。DNSに名前を登録しないことだけでは遮断になりません。TLSの証明書警告が出る場合は無視せず修正してください。

```
sudo ln -s /etc/nginx/sites-available/kptc-origin \
    /etc/nginx/sites-enabled/kptc-origin
sudo nginx -t
sudo systemctl reload nginx
curl -I https://ORIGIN_FQDN/
curl -I https://ORIGIN_FQDN/backup-scheduler-cli.php
```

正常ならNginx設定検査はsuccessful、トークンなしの予定表は403、CLIのURLは404です。403は今回の入口保護の正常動作です。200の予定表表示は12章の社内ポータル経由で確認します。

すでに同名のsites-enabledリンクがある場合は新規lnを繰り返さず、既存の指す先を確認します。PHPのソースが文字として表示される場合は直ちに公開停止し、PHP設定を見直してください。

9 tamanegi : 完成物とデータ領域

tamanegiで実行します。こちらには内部DBもバックアップJSONも転送しません。

```
cd ~/kptc-upload
grep '^.* tamanegi/' SHA256SUMS | sha256sum -c -
sudo install -d -m 755 /opt/kptc-scheduler/public/releases
sudo mkdir /opt/kptc-scheduler/public/releases/kptc-20260904
sudo rsync -a tamanegi/ /opt/kptc-scheduler/public/releases/kptc-20260904/
sudo chown -R root:www-data /opt/kptc-scheduler/public/releases/kptc-20260904
sudo find /opt/kptc-scheduler/public/releases/kptc-20260904 \
  -type d -exec chmod 750 {} \;
sudo find /opt/kptc-scheduler/public/releases/kptc-20260904 \
  -type f -exec chmod 640 {} \;
sudo ln -s /opt/kptc-scheduler/public/releases/kptc-20260904 \
  /opt/kptc-scheduler/public/current
sudo install -d -o www-data -g www-data -m 700 /var/lib/kptc-availability
sudo install -d -o root -g www-data -m 750 /etc/kptc-availability
```

正常なら照合OK、currentが新しい版を指します。公開JSONは初回送信時に自動生成されるため、この時点で無くても構いません。さくらの公開JSONを手で持ち込む必要はありません。

10 tamanegi : 非公開設定とPHP

sudo nano /etc/kptc-availability/public-env.php で保存。送信鍵はoriginと同じ実値にします。

```
<?php
putenv('KPTC_PUBLIC_AVAILABILITY_JSON=/var/lib/kptc-availability/public-availability.json');
putenv('KPTC_PUBLIC_AVAILABILITY_SECRET=REPLACE_PUBLISH_SECRET');
putenv('KPTC_PUBLIC_AVAILABILITY_STALE_SECONDS=1800');
```

```
sudo chown root:www-data /etc/kptc-availability/public-env.php
sudo chmod 640 /etc/kptc-availability/public-env.php
sudo php -l /etc/kptc-availability/public-env.php
```

sudo nano /etc/php/8.3/fpm/pool.d/kptc-tamanegi.conf で保存 :

```
[kptc-tamanegi]
user = www-data
group = www-data
listen = /run/php/php8.3-fpm-kptc-tamanegi.sock
listen.owner = www-data
listen.group = www-data
pm = ondemand
pm.max_children = 8
clear_env = yes
env[KPTC_PUBLIC_CONFIG_FILE] = /etc/kptc-availability/public-env.php
```

```
sudo php-fpm8.3 -t
sudo systemctl restart php8.3-fpm
```

正常なら設定検査successful。外部側にDB設定を追加しないでください。

11 tamanegi : NginxとHTTPS

tamanegi用証明書を同様に配置し、`sudo nano /etc/nginx/sites-available/kptc-tamanegi` で以下を保存します。

```
server {
    listen 80;
    server_name TAMANEGI_FQDN;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name TAMANEGI_FQDN;
    ssl_certificate /etc/ssl/kptc/tamanegi-fullchain.pem;
    ssl_certificate_key /etc/ssl/kptc/tamanegi-key.pem;
    root /opt/kptc-scheduler/public/current;
    index index.html;
    location / { try_files $uri $uri/ =404; }
    location ~ ^/(receive-availability|public-availability|health-availability)\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm-kptc-tamanegi.sock;
    }
    location ~ \.php$ { return 404; }
    location ~ /\. { deny all; }
}
```

```
sudo ln -s /etc/nginx/sites-available/kptc-tamanegi \
/etc/nginx/sites-enabled/kptc-tamanegi
sudo nginx -t
sudo systemctl reload nginx
curl -I https://TAMANEGI_FQDN/
curl -i https://TAMANEGI_FQDN/health-availability.php
```

正常ならトップ200。初回送信前のhealth 503は想定内です。証明書の更新は組織の管理方法に従ってください。

12 社内サイトの入口と一般モード

実運用では既存の社内サイトの認証後に、02_release/renkon/open-scheduler.php相当の処理を組み込みます。ログイン済み利用者の3桁IDを組織側で決定してください。利用者が任意IDを入力できるrenkonは模擬用で、本番本人確認には使いません。

項目	連携仕様
入力	user_001 のように user_ + 半角数字3桁
暗号	AES-256-CBC、元鍵をhash(sha256, 元鍵, true)で32バイト化
IV	毎回ランダム16バイト。IVと暗号文を連結してBase64化
転送	https://ORIGIN_FQDN/?token= にURLエンコードした結果を付ける
設定	両側のKPTC_PORTAL_TOKEN_KEYを一致させる。模擬renkonの内部URLはKPTC_RENKON_SCHEDULER_URLで指定
公開リンク	tamanegi URLは認証トークン不要。模擬renkonではconfig.jsで変更

正常なら社内サイトから一般モードの予定表が開きます。新規構築ではデモデータが作成されるため、本番利用前に管理者モードで内容を確認・整理します。旧環境のデータを使用する場合は13章を先に完了してください。

CBC形式だけでは改ざん検出・期限・本人との照合はありません。鍵の保護、認証済みの社内入口、社内ネットワーク制限をセットで運用します。URLに実名やパスワードを含めないでください。

13 既存データの移行と初期管理者

過去予定も含めて移行する場合は旧環境のSQLiteを正規の.backupで複製します。運用中の.sqliteだけをcpするとWAL内の変更を取りこぼす可能性があります。旧環境への予定入力と定期送信を停止し、移行用コピーが完成するまで再開しないでください。

```
sqlite3 /PRIVATE_PATH/group-watcher.sqlite \  
    ".backup '/PRIVATE_PATH/migration.sqlite'"  
sqlite3 /PRIVATE_PATH/migration.sqlite 'PRAGMA integrity_check;'
```

PRIVATE_PATHは旧環境の実際の非公開パスへ置換します。結果がokであることを確認し、migration.sqliteをSSH/SFTPでoriginの非公開作業領域へ転送します。所有者www-data・権限600で/var/lib/kptc-schedulerへ配置し、既存ファイルへ上書きせず、5章のDB設定を移行ファイルの絶対パスへ変更してください。JSONのみから移す場合は17章の復元手順を使います。

originでDBを準備した後、初期管理者モードのパスワードを設定します。初期設定が既にある移行DBでは、変更が必要な場合だけ行います。

```
sudo -u www-data env \  
    KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php \  
    php /opt/kptc-scheduler/internal/current/manage-auth-user-cli.php \  
    set-admin-mode-password
```

8～128文字を対話入力します。以後は画面の管理者モードで変更できます。JSONには設定ファイル・鍵・画像は含まれないため、別途移行します。

参照：SQLite公式 <https://www.sqlite.org/backup.html>

14 連携の手動試験

originで実行します。tamanegiへのHTTPS接続、同じHMAC鍵、両サーバーの時刻同期が必要です。

```
sudo -u www-data env \  
  KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php \  
  php /opt/kptc-scheduler/internal/current/publish-availability-cli.php \  
echo $?  
curl -i https://TAMANEGI_FQDN/health-availability.php
```

正常なら送信の終了コード0、healthはHTTP 200。公開画面でm6・ m7・ m8の当月から3か月を確認します。社内予定表に確認用予約を1件登録し、表示を確認後にその確認用予定を削除してください。

試験	合格条件
13～17時だけ機器利用	▲：午前のみ予約可
9～12時だけ機器利用	▼：午後のみ予約可
両時間帯の機器利用	濃灰、日付黒、記号なし
機器点検	一、日付なし
公開情報	個人名、件名、メモが含まれていない

origin側の試験に失敗したら、バックアップ定期実行を有効にする前に解決してください。公開JSONは内部DBの代用にも復元用にも使えません。

15 origin : 再送・監視・バックアップを登録

`sudo nano /etc/kptc-scheduler/internal-server.env` で以下の1行を保存します。すべてのCLIがWebと同じPHP設定を読み、秘密やDBパスの二重管理を避けます。既存設定を移す場合は古い重複値を整理してから使用してください。

```
KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php
```

```
sudo chown root:root /etc/kptc-scheduler/internal-server.env
sudo chmod 600 /etc/kptc-scheduler/internal-server.env
sudo cp ~/kptc-upload/deploy/kptc-availability-publish.service /etc/systemd/system/
sudo cp ~/kptc-upload/deploy/kptc-availability-publish.timer /etc/systemd/system/
sudo cp ~/kptc-upload/deploy/kptc-availability-monitor.service /etc/systemd/system/
sudo cp ~/kptc-upload/deploy/kptc-availability-monitor.timer /etc/systemd/system/
sudo cp ~/kptc-upload/deploy/kptc-scheduler-backup.service /etc/systemd/system/
sudo cp ~/kptc-upload/deploy/kptc-scheduler-backup.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now kptc-availability-publish.timer
sudo systemctl enable --now kptc-availability-monitor.timer
sudo systemctl enable --now kptc-scheduler-backup.timer
systemctl list-timers --all 'kptc-*
```

正常なら送信5分、監視10分、バックアップ日本時間22時の次回実行が表示されます。バックアップtimerはPersistent=trueのため、停止中に22時を過ぎた場合は次の起動後に実行します。

Ubuntuのserviceはwww-data、/usr/bin/php、/opt/.../internal/current、/etc/kptc-scheduler/internal-server.envを使用します。別配置にする場合は同梱serviceのパスも変更します。

16 初回バックアップと日常確認

originで手動実行して成功を確認します。自動実行の設定と同じserviceを使うため、権限や保存先も合わせて試験できます。

```
sudo systemctl start kptc-scheduler-backup.service
sudo journalctl -u kptc-scheduler-backup.service -n 20 --no-pager
sudo ls -l /var/lib/kptc-scheduler/backups/scheduler-latest.json
systemctl list-timers --all kptc-scheduler-backup.timer
```

正常ならログのokがtrue、fromDateが日本時間の実行日、schedulesが保存件数です。ファイルは所有者www-data、権限-rw-----。翌日の実行後にcreatedAtの更新も確認してください。登録だけでは自動実行の成功を証明しません。

当日以降に終了する予定を全件保存し、未来の期限はありません。継続中の複数日予定も含みます。ユーザー・試験室・種別・互換アカウント・管理者パスワードハッシュも保存します。過去予定・履歴は対象外ですが本番DBからは消しません。

保存は毎回1ファイルへ安全に置換します。壊れたDBや不正データでは旧正常JSONを保持します。公開用JSONではなく機密のバックアップなので、tamanegiやGitHubへ置かないでください。.lockは排他制御に必要で、削除しません。

同じサーバー自体の故障、正常な操作による誤削除の翌日保存、意味上の誤りには限界があります。別媒体への退避は組織で別途実施してください。メール通知・自動別媒体複製は本機能にありません。

17 障害復旧：最初に停止して保護

実行場所：origin。利用者へ停止を連絡し、社内ポータルからの新規アクセスを一時停止します。以下は専用サーバー用です。共用サーバーでは全PHP停止を行わず、担当者が対象poolと経路だけを停止してください。

```
sudo systemctl stop kptc-scheduler-backup.timer
sudo systemctl stop kptc-availability-publish.timer kptc-availability-monitor.timer
sudo systemctl stop kptc-scheduler-backup.service
sudo systemctl stop kptc-availability-publish.service kptc-availability-monitor.service
sudo systemctl stop php8.3-fpm
sudo systemctl is-active php8.3-fpm
sudo ls -l /var/lib/kptc-scheduler/backups/scheduler-latest.json
```

正常ならPHPはinactive。バックアップの更新日時が復元したい時点であることを確認します。異常の原因が分からない間はbackupを再実行しないでください。翌日の置換で復旧元を失う恐れがあります。

次に、元DB、-wal / -shmがあればそれら、設定、バックアップJSONを非公開の障害保全先へコピーします。下記の場所は新規名であることを確認し、既存なら別名を使います。障害保全用コピーは通常の日次世代とは別の臨時保全です。

```
sudo mkdir -m 700 /var/lib/kptc-scheduler/recovery-20260904
sudo cp -p /var/lib/kptc-scheduler/backups/scheduler-latest.json \
/var/lib/kptc-scheduler/recovery-20260904/
sudo cp -p /etc/kptc-scheduler/internal-env.php \
/var/lib/kptc-scheduler/recovery-20260904/
```

元DBパスはinternal-env.phpの実値を確認して保全してください。元DBは削除せず、この後は新しいファイルへ復元します。

18 JSONから新しいSQLiteを作成

originで実行。復元先group-watcher-restored-20260904.sqliteがまだ存在しないことを確認してください。同名があれば別の未使用名に変えます。元のgroup-watcher.sqliteを指定しません。

```
sudo -u www-data php \  
  /opt/kptc-scheduler/internal/current/restore-scheduler-cli.php \  
  /var/lib/kptc-scheduler/backups/scheduler-latest.json \  
  /var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite  
echo $?  
sudo -u www-data sqlite3 \  
  /var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite \  
  'PRAGMA integrity_check;'  
sudo -u www-data sqlite3 \  
  /var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite \  
  "SELECT json_array_length(payload,'$.schedules'),version FROM app_state;"
```

正常なら「本番DBは変更していません」と表示され、終了コード0、整合性はokです。最後の行は予定件数と更新世代です。復元はバックアップのJSON形式・照合値を確認してから実施され、既存ファイルへの上書きは拒否します。

この時点では新しいDBを作ただけで、予定表はまだ元DB設定のままです。JSONは手で編集しないでください。公開側のpublic-availability.jsonには復元に必要な予定情報がないため使用できません。

復元できるのはバックアップ作成時点の対象予定・マスター・アカウント・管理者設定です。過去に終了した予定と操作履歴は戻りません。期間を過ぎてから復元しても、JSONに含まれる内容をそのまま戻します。

19 内容確認と公開世代の調整

originの復元DBを確認します。以下の表示には個人名や予定件名を出さず、件数だけを確認します。

```
sudo -u www-data sqlite3 \  
/var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite \  
"SELECT json_array_length(payload,'$.members'),  
        json_array_length(payload,'$.categories') FROM app_state;"  
sudo -u www-data sqlite3 \  
/var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite \  
'SELECT count(*) FROM auth_users; SELECT count(*) FROM audit_logs;'
```

正常ならユーザー・種別・互換アカウント件数が想定どおり、履歴は0件。管理者パスワードは保存時のものへ戻ります。

tamanegiで現在の公開JSONのsourceVersionを読みます。公開側を削除して解決しないでください。

```
sudo -u www-data php -r \  
'$p=json_decode(file_get_contents($argv[1]),true,512,JSON_THROW_ON_ERROR);  
echo $p["sourceVersion"],PHP_EOL; ' \  
/var/lib/kptc-availability/public-availability.json
```

originへ戻り、復元DBのversionと公開sourceVersionの大きい方に1を加えた値を使用します。例：復元100、公開125なら126。次の126は実際の計算値に置き換えてください。公開JSONが初回未生成なら復元version + 1でよいです。

```
sudo -u www-data sqlite3 \  
/var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite \  
'UPDATE app_state SET version=126 WHERE id=1;'
```

これは復元直後の古い世代が外部受信で拒否されないための処置です。作業中は送信処理を停止したままにします。

20 設定を切替・公開情報を再生成

originで `sudo nano /etc/kptc-scheduler/internal-env.php` を開き、DB指定の1行だけを新規復元DBへ変更します。他の秘密設定を変更しません。

```
putenv('KPTC_INTERNAL_SCHEDULER_DB=/var/lib/kptc-scheduler/group-watcher-restored-20260904.sqlite');
```

長い行ですが、PHP設定では1行として保存します。既存セッションを持ち越さないよう、専用の新しい空フォルダーを用意します。

```
sudo install -d -o www-data -g www-data -m 700 \  
/var/lib/kptc-scheduler/sessions-restored-20260904  
sudo nano /etc/php/8.3/fpm/pool.d/kptc-origin.conf
```

pool内の`session.save_path`の行を以下へ変更します。元の行と元DBパスを切戻し用に控えておきます。

```
php_admin_value[session.save_path] = /var/lib/kptc-scheduler/sessions-restored-20260904
```

```
sudo php -l /etc/kptc-scheduler/internal-env.php  
sudo php-fpm8.3 -t  
sudo -u www-data env \  
KPTC_INTERNAL_CONFIG_FILE=/etc/kptc-scheduler/internal-env.php \  
php /opt/kptc-scheduler/internal/current/publish-availability-cli.php  
curl -i https://TAMANEGI_FQDN/health-availability.php  
sudo systemctl start php8.3-fpm
```

正常なら送信終了コード0、health 200。利用者は社内ポータルから入り直します。公開側に古い更新世代エラーが出たら19章を再確認し、失敗したまま再開しないでください。

21 復旧確認・定期処理再開・切戻し

社内ポータル経由で予定表を開き、予定・ユーザー・試験室・種別を確認します。管理者モードへ保存時のパスワードで切替できること、確認用予定の登録・編集・削除ができること、空き状況へ反映されることを確認します。確認用予定は終了後に削除します。

```
sudo systemctl start kptc-availability-publish.timer
sudo systemctl start kptc-availability-monitor.timer
sudo systemctl start kptc-scheduler-backup.timer
systemctl list-timers --all 'kptc-*'
sudo journalctl -u kptc-availability-publish.service -n 20 --no-pager
```

バックアップtimerは取り逃し分を直ちに実行することがあります。復旧元を保全した17章が完了し、復元内容が正しいと確認した後に再開してください。通常の最新JSONは次の成功時に置換されます。

問題がある場合：再び入力と3つのtimer/service、PHPを停止し、internal-env.phpのDBパスを元の非公開DBへ戻します。元DB自体が破損している場合は戻らず停止を継続してください。復旧中に公開情報を更新した場合、元DBにも19章の世代調整を行ってから再送します。

新しい空のsession.save_pathはそのまま使い、旧セッションを復活させないでください。設定検査→再送→画面確認→定期処理再開の順です。復元後に入力された変更は元DBへ自動では戻らないため、切戻し前に担当者が扱いを決めます。

元DB・障害保全コピーは障害原因と復旧結果が確定するまで削除しません。通常の「日次バックアップは1世代」と、事故時の一時保全は分けて扱います。

22 さくら環境：現状と復元時の違い

2026年9月4日時点で下記を登録済みです。サーバー時刻はJST。既存の送信・監視は変更せず、バックアップを追加しています。

```
* /5 * * * * /home/apfelrunner/GW/bin/publish-availability \
>/home/apfelrunner/GW/publish-availability.log 2>&1
* /10 * * * * /home/apfelrunner/GW/bin/monitor-availability \
>/home/apfelrunner/GW/monitor-availability.log 2>&1
0 22 * * * /usr/local/bin/php /home/apfelrunner/www/GW/schedule/backup-scheduler-cli.php \
>/home/apfelrunner/GW/scheduler-backup.log 2>&1
```

上の枠は読みやすく折り返しています。crontabでは各ジョブを改行せず1行で登録します。既に登録済みなので重複追加は不要です。初回手動実行は23件・8,036バイト、保存・復元検証成功。翌日以降はログと更新日時を確認します。

用途	実パス
実DB	/home/apfelrunner/GW/group-watcher.sqlite
バックアップ	/home/apfelrunner/GW/backups/scheduler-latest.json
内部設定	/home/apfelrunner/GW/config/internal-env.php
復元CLI	/home/apfelrunner/www/GW/schedule/restore-scheduler-cli.php
PHP	/usr/local/bin/php

さくらではsystemctlを使いません。利用停止・対象cronの一時コメント化・進行中処理の終了確認を行い、上記CLIへJSONと未使用の非公開SQLiteパスを指定します。内容検証・世代調整・DB設定切替は17～21章と同じ考え方です。共有PHP全体は停止せず、対象サイトだけの停止方法を管理者と確認してください。

23 更新・試験室追加・画像差替

更新は新しいreleases/版名へ完成物一式を転送し、照合・PHP検査後にcurrentを切り替えます。旧版と設定・DBはそのまま残し、公開フォルダーと非公開データ領域をまとめて削除しないでください。

```
sudo ln -sf /opt/kptc-scheduler/internal/releases/NEW_RELEASE \
/opt/kptc-scheduler/internal/current
sudo systemctl reload php8.3-fpm
```

NEW_RELEASEは実際に配置済みのフォルダー名へ置換します。戻す場合はcurrentを記録済みの旧版へ変更します。コードの切替だけではDBは元に戻りません。

試験室追加：①管理者モードで所属「試験室」のユーザー作成、②availability-room-config.phpの許可IDに追記、③必要な公開名・説明を設定、④originへ設定PHPを反映、⑤tamanegiへID.pngを配置、⑥手動送信と公開画面確認。公開許可と画像使用許可を先に得てください。

画像は同名ファイルを差し替えれば変更できます。m6.png・m7.png・m8.pngは現行200×200ピクセル。縦横比を揃えた正方形を基本とし、重要部分を中央へ置きます。ブラウザのキャッシュで古い画像が見える場合は再読込してください。

新releaseへ切替える前に、サーバーだけにある追加室画像・差替画像を引き継ぎます。既存のユーザー作成画像をビルド画像で意図せず上書きしないよう、差分を見て選びます。秘密設定・鍵・画像の別媒体保管は日次JSONとは別に必要です。

24 受入確認と困ったとき

確認 / 症状	合格条件・最初の確認
入口	トークンなし403、正規社内入口で予定表、CBC鍵一致。
管理	一般は予定編集可、管理者パスワードで管理項目が使える。
予定操作	今週、7日表示、休日、長い室名、D&Dの確認・取消、複数操作。
公開連携	正しい3か月・許可室・記号、個人情報なし、health 200。
定期処理	送信・監視・バックアップの3timerを確認し、実行後のログも確認。
バックアップ	private JSONの時刻・件数・600権限。別名SQLiteへの復元がok。
PHP/画面不調	nginx -t、php-fpm8.3 -t、サービス、socket、index.php/HTMLとassets。
保存不可	DBと親フォルダーのwww-data権限、空き容量、APIの409競合。
送信失敗/503	URL、HMAC鍵、時刻差、送信ログ、JSONの鮮度・権限。
復元失敗	JSON種類・照合値、既存復元先、SQLite拡張、書込権限。前回正常JSONは編集しない。
復元後409	公開sourceVersionより復元DBのversionが古くないか19章で確認。

```
sudo journalctl -u php8.3-fpm -n 30 --no-pager
sudo journalctl -u kptc-availability-monitor.service -n 30 --no-pager
sudo journalctl -u kptc-scheduler-backup.service -n 30 --no-pager
```

本書のUbuntu構成は配置用の例です。コード・復元機能はローカルとさくらで検証済みですが、origin/tamanegi実機での新規構築・受入試験は現地担当者が実施してください。

25 用語集：機械・通信・公開

言葉	やさしい説明
サーバー	ほかの人からの要求を受け、画面や情報を渡すコンピューター。
内部 / 外部	組織の中だけで使う範囲 / インターネットから見られる範囲。
OS・Ubuntu・Linux	機械全体を動かす基本のソフト。UbuntuはLinuxの仲間。24.04は版の名前。
ブラウザー	SafariやChromeなど、Webページを見るソフト。
URL・ドメイン・FQDN	Webの場所を示す宛先 / 機械の読みやすい名前 / 省略しない正式名。
IPアドレス・DNS	機械の数字の住所 / 名前から数字の住所を調べる住所録。
LAN・VPN	近くの機械を結ぶ網 / 離れた場所から組織の網へ安全につなぐ道。
ポート・443・22	用途別の入口番号。443は暗号化したWeb、22は遠隔操作で主に使う。
HTTPS・TLS・証明書	通信を読まれにくくし相手確かめる仕組み / その仕組み / 相手の電子的な名札。
SSH・SFTP	離れた機械を安全に操作する方法 / 安全にファイルを送る方法。
Nginx・PHP-FPM	Web要求の受付係 / PHPで書かれた処理を実行する係。
socket・pool	プログラム同士をつなぐ連絡口 / PHPの仕事を担当するまとめり。

25 用語集：ファイル・設定・操作

言葉	やさしい説明
ターミナル・ シェル・ Bash	文字で命令する画面 / 命令を読む係 / その係の一種。
コマンド・ CLI	文字で書く命令 / 文字の命令で使う入口。
sudo・ root・ www-data	管理者の力で実行する命令 / 管理者 / Web処理用の利用人名。
権限600・ 700・ 640	600は所有者だけ読書き、700は所有者だけ使えるフォルダー、640は所有者が読書きし指定の仲間は読むだけ。
フォルダー・ パス	ファイルをまとめる入れ物 / その入れ物やファイルの場所を表す文字列。
環境変数・ 設定ファイル	場所ごとに変える保存先や宛先などの値 / その値をまとめたファイル。
アップロード・ rsync・ scp	ファイルを送ること / 差を確かめて送る道具 / 安全にコピーして送る道具。
ソース・ TS/TSX・ React	人が編集する元のプログラム / 画面の動きを書く言葉 / 画面部品を作る仕組み。
ビルド・ Vite・ Node.js・ pnpm	元を実行用に作り替えること / 変換の道具 / 開発用処理を動かす道具 / 必要部品をそろえる道具。
release・ current・ リンク	配布する版 / 今使う版の目印 / 別の場所へ案内する近道。
GitHub・ main	プログラムの変更を保管するサービス / この案件で正式版をまとめる場所。
キャッシュ	前に受け取った画像などを近くに保存して、次を速くする仕組み。

25 用語集：データ・認証・復旧

言葉	やさしい説明
DB・SQLite・テーブル	整理して情報を置く箱 / ファイルで管理するDB / DB内で同じ種類をまとめた表。
JSON・schema	名前と値を文字で並べる保存形式 / その形式の決まり。
バックアップ・復元	予備の写しを作ること / 写しを使って使える状態へ戻すこと。
整合性・SHA-256	データのつじつまが合うこと / 内容から作る照合用の数字と文字。
原子的置換・ロック	完成品を一度で入れ替える方法 / 同時に同じ作業をしないための使用中の印。
WAL・トランザクション	DB本体に反映前の変更記録 / 複数の処理をまとめて成功が失敗かにする単位。
version・sourceVersion	更新の順番を表す番号。新旧を取り違えないために使う。
CBC・IV・Base64	暗号の作り方 / 毎回変える暗号用の出発値 / データを文字で運べる形にする方法。
token・セッション・Cookie	入口へ渡す文字情報 / 利用中の状態を覚える仕組み / ブラウザーが持つ小さな目印。
HMAC・共通鍵・CSRF	秘密を共有する相手の印 / 双方が持つ秘密の文字列 / 別サイトから本人の意図なしに操作させる攻撃。
cron・systemd・timer・service	時刻で動かす仕組み / Ubuntuの仕事管理係 / 開始の時計 / 仕事本体。
ログ・終了コード・切戻し	起きたことの記録 / 成功失敗を示す番号 / 問題前の構成へ戻すこと。

迷ったら、どの機械で、どのファイルを、何のために変更するのかを確認してください。値が分からないまま命令を実行しないことが、安全な復旧への近道です。